

CTIT: Integration Testing for Clang-Tidy

Zeyi Xu, Victor Baranov

Background

A clang-tidy change may break:

Accuracy

False positives, false negatives, invalid fixes

Robustness

Crashes on valid code

Performance

Analysis time explosion at project scale

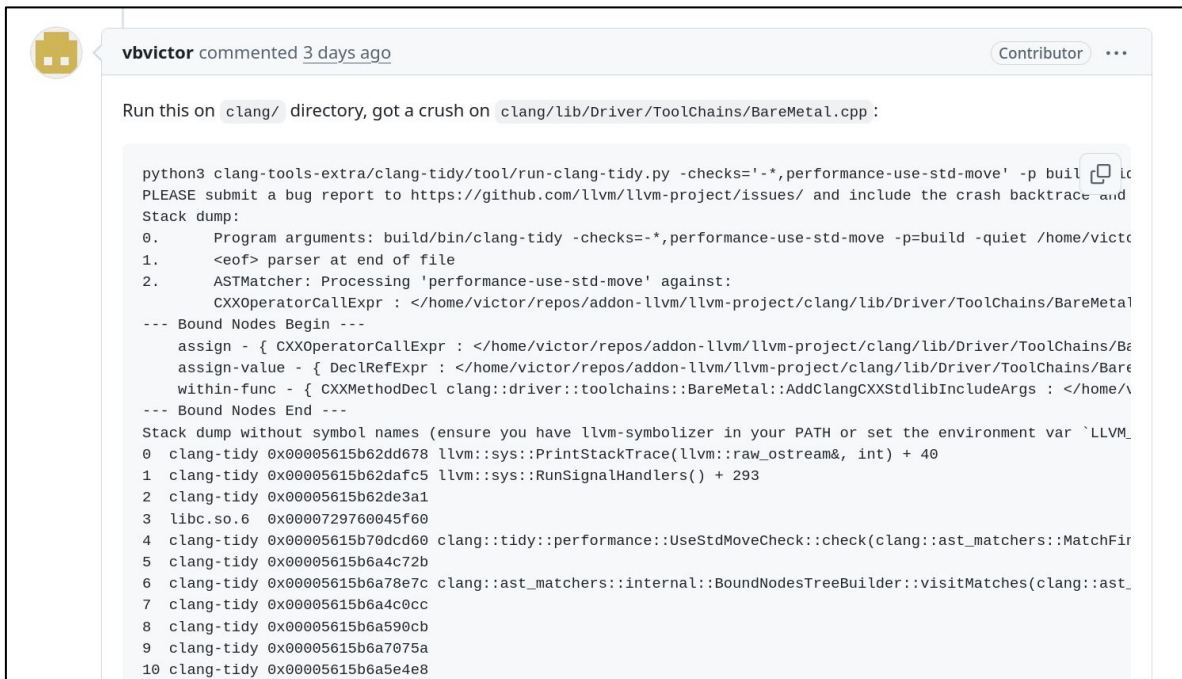
Previous Solution:

- **PR Author's Self Check**
- **Code Reviewers' Review**
- **Users Bug report**

From Clang-Tidy contributing guide:

“To test a check, it’s best to try it out on a larger code base.”

In Reality... Some PR Authors don’t do it themselves.



The screenshot shows a GitHub comment interface. At the top left is a yellow house icon. To its right, the text "vbvictor commented 3 days ago" is displayed. On the far right of the comment header is a button labeled "Contributor" followed by three dots. The main body of the comment contains a text block starting with "Run this on clang/ directory, got a crush on clang/lib/Driver/ToolChains/BareMetal.cpp :". Below this is a large code block with a light gray background. The code is a terminal output showing the execution of clang-tidy. It includes a header for submitting bug reports, a list of program arguments, a stack dump, and a list of AST nodes. The stack dump shows a sequence of function calls from clang-tidy to libc.so.6, ending with a crash in the clang-tidy process.

```
python3 clang-tools-extra/clang-tidy/tool/run-clang-tidy.py -checks=*,performance-use-std-move' -p build
PLEASE submit a bug report to https://github.com/llvm/llvm-project/issues/ and include the crash backtrace and
Stack dump:
0. Program arguments: build/bin/clang-tidy -checks=*,performance-use-std-move -p=build -quiet /home/victor
1. <eof> parser at end of file
2. ASTMatcher: Processing 'performance-use-std-move' against:
   CXXOperatorCallExpr : </home/victor/repos/addon-llvm/llvm-project/clang/lib/Driver/ToolChains/BareMetal
--- Bound Nodes Begin ---
   assign - { CXXOperatorCallExpr : </home/victor/repos/addon-llvm/llvm-project/clang/lib/Driver/ToolChains/Bare
   assign-value - { DeclRefExpr : </home/victor/repos/addon-llvm/llvm-project/clang/lib/Driver/ToolChains/Bare
   within-func - { CXXMethodDecl clang::driver::toolchains::BareMetal::AddClangCXXStdlibIncludeArgs : </home/v
--- Bound Nodes End ---
Stack dump without symbol names (ensure you have llvm-symbolizer in your PATH or set the environment var `LLVM_
0 clang-tidy 0x00005615b62dd678 llvm::sys::PrintStackTrace(llvm::raw_ostream&, int) + 40
1 clang-tidy 0x00005615b62dafc5 llvm::sys::RunSignalHandlers() + 293
2 clang-tidy 0x00005615b62de3a1
3 libc.so.6 0x0000729760045f60
4 clang-tidy 0x00005615b70dcd60 clang::tidy::performance::UseStdMoveCheck::check(clang::ast_matchers::MatchFir
5 clang-tidy 0x00005615b6a4c72b
6 clang-tidy 0x00005615b6a78e7c clang::ast_matchers::internal::BoundNodesTreeBuilder::visitMatches(clang::ast_
7 clang-tidy 0x00005615b6a4c0cc
8 clang-tidy 0x00005615b6a590cb
9 clang-tidy 0x00005615b6a7075a
10 clang-tidy 0x00005615b6a5e4e8
```

Lit Tests

```
// RUN: %check_clang_tidy %s bugprone-signal-handler %t
// RUN:  -- -- -isystem %S/../../Inputs/Headers

#include <signal.h>          // mock system header

void test_handlers(void) {
    signal(SIGINT, other_call);
    // CHECK-NOTES: warning: standard function 'other_call' may not be asynchronous-safe
}
```

Small, purpose-written fixtures

Fast, deterministic, portable

**But they only cover what authors
and reviewers anticipate**

As of February 16, 2026, 92 out of 611 clang-tidy checks still lack comprehensive test coverage for their configuration options.

On Clang-Tidy Issues

From Jan 2021 – Sep 2026:

- 673 Issues that are FP / FN / Crash / Invalid fix-its

546 of them trace back to conditions the focused tests didn't cover.

Mechanism	Reports	FP	FN	Invalid fix	Crash
Ecosystem drift New standards and library versions	47	22	2	7	17
Unanticipated context Language, macro and build setups nobody tested	493	260	84	68	87
Project-scale amplification Small defects multiplied by project size	6	1	0	2	4

Examples: Ecosystem Drift

clang-tidy: Getting "misc-static-assert" reports for every assert #213560 

 Open

 ryan0270 opened on Aug 3 

Summary

clang-tidy is generating this warning for **every** assert, even ones that obviously cannot be evaluated at compile time.

Steps to reproduce

repro.cpp :

```
#include <cassert>

void f(int* p, int n)
{
    assert(p != nullptr);
    assert(n > 0);
    assert(p);
}
```



Trace back to:

<https://sourceware.org/git/?p=glibc.git;a=commit;h=ea37298b65bd67f94c3c2640e91ec5865a5019ad>

Example: Unanticipated Context

[clang-tidy] Crash with readability-non-const-parameter and K&R declarations #100340 

✓ Closed

 #100461



mkmvzlb opened on Jul 24, 2024

Contributor



The simple legacy C-code below crashes clang-tidy, at least version 10, 12 and 16:

```
static int f();  
  
int f(p)  
  int *p;  
{  
    return *p;  
}
```



The tests didn't cover K&R before.

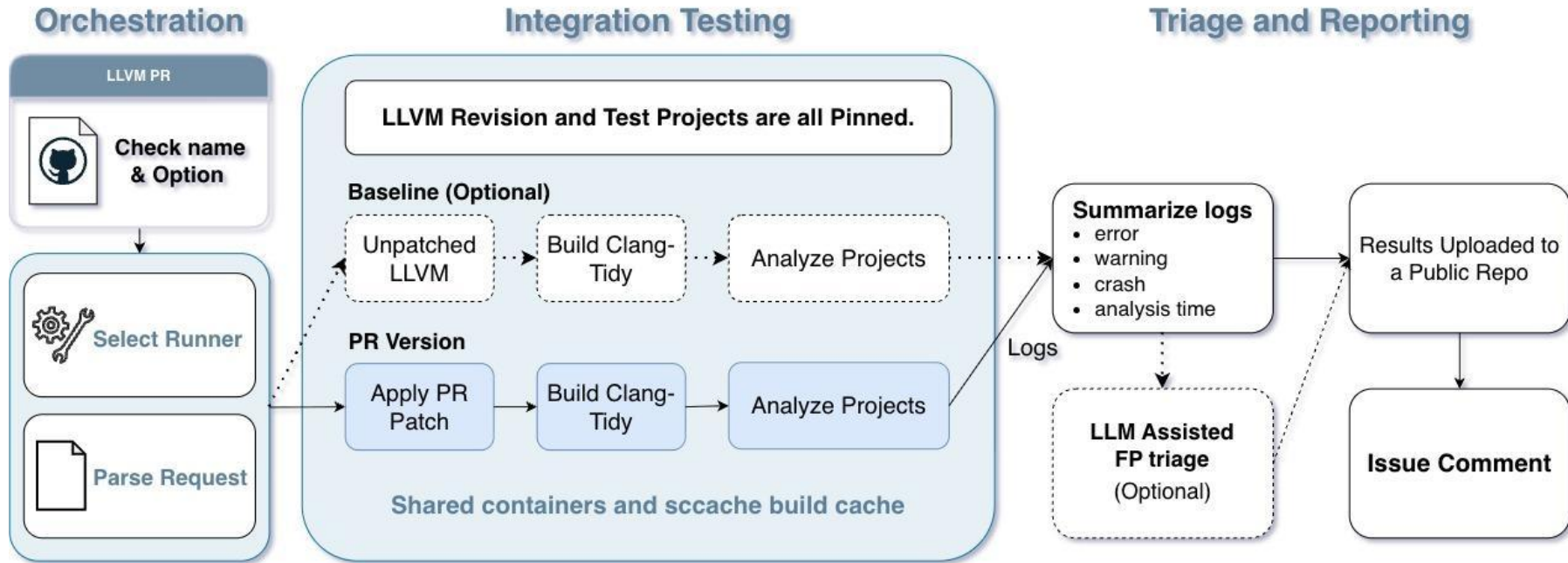
Existing Tools?

- Clippy Lintcheck: Very close to what we want, but it is designed for Rust.
- Codechecker: It uses real projects, but its goal is to test the projects themselves. We need the exact opposite: using real projects to test the linter.

So we need a new tool that:

- Tests clang-tidy on diverse real-world projects
- Easy to use and maintain, open to every contributors
- Review native, everything happens on Github

Architecture (in a simple way)



Usage

Test a check from an LLVM pull request

Who can start a run

Runs start only for users with commit access to `llvm/llvm-project`. If you do not have commit access, open the issue anyway and ask a clang-tidy maintainer or reviewer to comment `/redo` on it. Their comment starts the run.

Steps

1. [Open a new issue](#) using the **Integration test** template.
2. Replace the issue body with your request (see the format below).
3. Make sure the issue has the `cpp` or `c` label. The template adds `cpp` automatically. Adding the label is what starts the run.
4. Wait for the results. CTIT posts a "Run started" comment with a link to the workflow run, and replaces it with the report when the run finishes.

Issue body format

```
[PR_URL] CHECK_NAME  
[OPTION_NAME: VALUE]  
[OPTION_NAME: VALUE]  
[/baseline]
```



The projects we currently test

C++

Cppcheck

LLVM/Clang

Doxygen

POCO

Abseil

stdexec

Catch2

yaml-cpp

Assimp

C

curl

zstd

libuv

libgit2

***CTIT can test them with 3 std-lib profiles:**

- **libstdc++ 14 / 15, libc++ 21**

Is it useful?

Clang-Tidy Integration Test Results

Project	Status	Warnings	Errors	Crash
abseil-cpp	⚠ Warnings	418	0	-
cppcheck	⚠ Warnings	40	0	-
curl	⚠ Warnings	1102	0	-
doxygen	⚠ Warnings	331	0	-
llvm-project	⚠ Warnings	204	0	-
poco	⚠ Warnings	989	0	-
stdexec	⚠ Warnings	53	0	-



Project	Status	Warnings	Errors	Crash	Full tidy time (s)
abseil-cpp	⚠ Warnings	3	0	-	11.14
assimp	⚠ Warnings	4	0	-	24.80
catch2	✅ Pass	0	0	-	4.85
cppcheck	✅ Pass	0	0	-	13.02
curl	⚠ Warnings	9	0	-	2.52
doxygen	⚠ Warnings	29	0	-	30.78
libgit2	⚠ Warnings	5	0	-	1.48
libuv	⚠ Warnings	2	0	-	0.88
llvm-project	⚠ Warnings	1	0	-	336.48
poco	⚠ Warnings	45	0	-	43.59
stdexec	✅ Pass	0	0	-	7.75
yaml-cpp	✅ Pass	0	0	-	2.90
zstd	✅ Pass	0	0	-	2.72

<https://github.com/clang-tidy-infra/CTIT/issues/183>

CTIT helped identify FP flood

Is it useful?

Clang-Tidy Integration Test Results

Project	Status	Warnings	Errors	Crash
abseil-cpp	💣 CRASH	0	0	YES
cppcheck	💣 CRASH	0	0	YES
curl	✅ Pass	0	0	-
doxygen	💣 CRASH	0	0	YES
llvm-project	💣 CRASH	0	0	YES
poco	💣 CRASH	0	0	YES
stdexec	💣 CRASH	0	0	YES



Clang-Tidy Integration Test Results

Project	Status	Warnings	Errors	Crash	Full tidy time (s)
abseil-cpp	✅ Pass	0	0	-	10.49
assimp	✅ Pass	0	0	-	18.52
catch2	✅ Pass	0	0	-	4.05
cppcheck	✅ Pass	0	0	-	12.80
curl	✅ Pass	0	0	-	2.54
doxygen	✅ Pass	0	0	-	29.49
libgit2	✅ Pass	0	0	-	1.52
libuv	✅ Pass	0	0	-	0.81
llvm-project	✅ Pass	0	0	-	328.70
poco	✅ Pass	0	0	-	44.42
stdexec	✅ Pass	0	0	-	8.24
yaml-cpp	✅ Pass	0	0	-	3.15
zstd	✅ Pass	0	0	-	2.86

<https://github.com/clang-tidy-infra/CTIT/issues/187>

CTIT helped identify crashes

Performance: a dataflow solver regression

(Found by Victor while he was working on Daily Time Checking feature)

LLVM translation unit	Before (s)	After (s)	Speedup
RISCVVEmitter.cpp	8235.7	14.2	580×
Multilib.cpp	8197.2	12.2	673×

- 99.9% of wall time in bugprone-unchecked-optional-access
- Cause: solver invariant checks rebuilt state on every iteration
- Fix: keep them only in expensive-checks builds (#205715)
- LLVM's own pre-merge CI used the same configuration

Replay Experiment

I used CTIT to check 163 clang-tidy PRs from two LLVM release cycles. Among the 121 modified / new checks, CTIT surfaced 15 FP regressions.

Outcome	20–21	21–22	Total
False-positive regressions	4	11	15
Later reported by users	3	8	11
Fixed without an issue	0	2	2
Still unreported	1	1	2

Operational cost

On average, it takes CTIT around 20mins to test a PR.

Slower checks might take longer (up to 90mins).

Merge lead time

Average time taken for pull requests to be raised, reviewed, accepted, and merged in the selected period. [Learn more](#)

Average time to merge

7 days  33.4% (4d)
vs. 11d last period

(<1% of the whole
reviewing process)

<https://insights.linuxfoundation.org/project/llvm-llvm-project/development>

Repo

Main repo: <https://github.com/clang-tidy-infra/CTIT>

Run results: <https://github.com/clang-tidy-infra/CTIT-results>

Future works

- False negatives: real-world cases with known expected diagnostics
- LLM FP Triage: Test more models (for quicker analysis / better accuracy)
- Wider corpus and configs: std libs, platforms, build systems
- Apply fixes and rebuild projects to catch invalid fixes
- Website for displaying and comparing test results

Concerns so far...

1. Is Clang Static Analyzer interested in using this tool?
2. What new features should this tool adopt?
3. New projects to test?